

Запитання: Лабораторна робота №1

(Тема: Об'єктно-орієнтоване програмування в C#, класи, наслідування та інтерфейси)

1. Що таке класи та об'єкти в C#? Яка між ними різниця?

Відповідь: Клас — це шаблон, креслення або тип даних, який описує структуру та поведінку майбутнього об'єкта (наприклад, наш клас `Product` описує, що у кожного товару має бути `Id`, `Name` та `Price`). Об'єкт — це конкретний екземпляр класу, створений у пам'яті комп'ютера за цим шаблоном за допомогою ключового слова `new` (наприклад, конкретний товар з `Id = 1` та іменем "Хліб").

2. Чим інтерфейс (Interface) відрізняється від звичайного класу?

Відповідь: Інтерфейс (наприклад, наш `ICrudServiceAsync<T>`) — це суто декларативна структура, яка не містить полів та реалізації методів (контракт). Він лише визначає, що саме повинен вміти робити клас, який його реалізує. Клас, навпаки, містить конкретну логіку та дані. Клас може наслідувати лише один інший клас, але може реалізовувати будь-яку кількість інтерфейсів.

3. Навіщо у властивостях використовуються `get` та `set`?

Відповідь: Це механізм інкапсуляції (аксесори). `get` дозволяє читати значення властивості ззовні класу, а `set` — записувати або змінювати його. Завдяки автоматичним властивостям (`public string Name { get; set; }`) ми уникаємо створення прихованих полів вручну, але зберігаємо контроль над доступом до даних.

Контрольні запитання: Лабораторна робота №2

(Тема: Асинхронне програмування, багатопотоковість та безпечні колекції)

1. Що таке асинхронність та ключові слова `async` / `await`?

Відповідь: Асинхронність дозволяє виконувати тривалі операції (наприклад, запис у базу даних чи генерацію даних) у фоновому режимі, не блокуючи головний потік програми. Ключове слово `async` вказує компілятору, що метод містить асинхронний код і повертає об'єкт типу `Task`. Ключове слово `await` тимчасово призупиняє виконання методу, поки фонові задача не завершиться, звільняючи потік для інших задач.

2. Чому для багатопотокової генерації використано `ConcurrentBag<T>`, а не звичайний `List<T>`?

Відповідь: Звичайний `List<T>` не є безпечним для потоків (*not thread-safe*). Якщо кілька потоків одночасно спробують записати в нього дані через `Parallel.For`, виникне стан гонитви (*race condition*), дані затруться або програма впаде з помилкою. `ConcurrentBag<T>` — це спеціальна колекція з простору імен `System.Collections.Concurrent`, яка має внутрішні механізми синхронізації (блокування) і дозволяє багатьом потокам безпечно й паралельно додавати об'єкти без збоїв.

3. Що робить метод `Parallel.For`?

Відповідь: Він запускає паралельний цикл, розподіляючи ітерації (у нашому випадку від 0 до 1000) між усіма доступними ядрами процесора.

Замість послідовного виконання в одному потоці процесор виконує генерацію замовлень паралельно, що значно прискорює роботу на багатоядерних системах.

Контрольні запитання: Лабораторна робота №3

(Тема: *ORM Entity Framework Core, шаблони Repository та Unit of Work, SQLite*)

1. Що таке ORM (Object-Relational Mapping) та Entity Framework Core?

Відповідь: ORM — це технологія, яка автоматично зв'язує таблиці реляційної бази даних з об'єктами (класами) у коді C#. Entity Framework Core — це офіційна ORM від Microsoft, яка дозволяє розробнику працювати з базою даних (shop.db) за допомогою звичних об'єктів C# та LINQ-запитів, взагалі не пишучи сирого SQL-коду вручну.

2. Яку роль відіграє клас DbContext та метод OnModelCreating?

Відповідь: DbContext (наш клас ShopContext) — це серце EF Core, яке координує роботу з БД, відстежує зміни в моделях та зберігає їх. Метод OnModelCreating використовується для налаштування конфігурації бази даних за допомогою **Fluent API**. Саме там ми явно вказали назви таблиць, налаштували стратегію наслідування та зв'язок "один-до-багатьох" між категоріями й товарами.

3. Що таке стратегія наслідування Table-per-Type (TPT) у базах даних?

Відповідь: Стратегія TPT створює в базі даних окрему таблицю для кожного класу в ієрархії наслідування. У нашому проєкті базові властивості товару зберігаються в таблиці Products, а специфічні властивості цифрового товару (DownloadUrl) — в окремій таблиці DigitalProducts. Вони пов'язані між собою через первинний ключ (Foreign Key). Це наочно видно на нашому скріншоті з *DB Browser*.

4. Навіщо потрібен шаблон проектування «Репозиторій» (Repository)?

Відповідь: Він створює рівень абстракції між бізнес-логікою програми та інфраструктурою бази даних. Замість того, щоб консольний додаток напряму звертався до контексту EF Core, він працює через OrderRepository. Це ізолює програму від конкретної СУБД (полегшує перехід з SQLite на SQL Server чи PostgreSQL) і запобігає дублюванню коду запитів.